DSCI 372M Homework 1

Steven Sanchez

January 2026

Question 1

Part (a):

$\hat{y} = w_0 + w_1x_1 + w_2x_2, Error = \hat{y} - y$

$\hat{y_1} = 0 + 0(1) + 0(1) = 0$, Error = 0 - (-0.8) = 0.8
$\hat{y_2} = 0 + 0(-1) + 0(-2) = 0$, Error = 0 - (0.1) = -0.1
$\hat{y_3} = 0 + 0(2) + 0(-1) = 0$, Error = 0 - (-5.0) = 5.0

Mean Squared Error: $\frac{1}{3}[(0.8)^2 + (-0.1)^2 + (5.0)^2] = 8.55$

$w_0 : \frac{2}{3}(0.8 - 0.1 + 5.0) = 3.8$, $w_0 = 0 - 0.05(3.8) = -0.19$
$w_1 : \frac{2}{3}(0.8(1) - 0.1(-1) + 5.0(2)) = 7.26$, $w_1 = 0 - 0.05(7.26) = -0.363$
$w_2 : \frac{2}{3}(0.8(1) - 0.1(-2) + 5.0(-1)) = -2.66$ $w_2 = 0 - 0.05(-2.66) = 0.133$

New weights: $w_0, w_1, w_2 = -0.19, -0.363, 0.133$

Iteration	w_0	w_1	w_2	MSE
1	-0.19	-0.36	0.13	8.55
2	-0.36	-0.65	0.24	5.26
3	-0.43	-0.87	0.32	3.31
4	-0.49	-1.04	0.39	2.15

```
[26]: x = np.array([
          [1, 1],
          [-1, -2],
          [2, -1]
      ])

      y = np.array([-0.8, 0.1, -5.0])

      w0, w1, w2 = 0.0, 0.0, 0.0
      alpha = 0.05
      n = len(y)

      for i in range(1, 5): #running 4 iterations
          y_hat = w0 + (w1 * x[:, 0]) + (w2 * x[:, 1])
          error = y_hat - y
          mean_squared_error = np.mean(error ** 2)
          gradient1 = (2/n) * np.sum(error)
          gradient2 = (2/n) * np.sum(error * x[:, 0])
          gradient3 = (2/n) * np.sum(error * x[:, 1])
          w0 = w0 - alpha * gradient1
          w1 = w1 - alpha * gradient2
          w2 = w2 - alpha * gradient3
          print(w0, w1, w2, mean_squared_error)

      -0.19 -0.3633333333333334 0.1333333333333333 8.549999999999999
      -0.3278888888888889 -0.6457777777777778 0.2394444444444446 5.261425925925926
      -0.4260851851851852 -0.8660777777777777 0.3245555555555556 3.31308090946502
      -0.4941011111111111 -1.0386083950617284 0.3934413580246914 2.1533203291495204
```

The code above shows the iteration for the next 3, but we showed the step by step process for the first one so we can visually see what is happening.

Part (b):

For this, we learned back in MA342: w = $(X^T X)^{-1} X^T Y$

Here, we have our matrices:

X = $\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & -2 \\ 1 & 2 & -1 \end{bmatrix}$ and Y = $\begin{bmatrix} -0.8 \\ 0.1 \\ -5.0 \end{bmatrix}$ where X^T is the transpose of X (basically flipping the rows and columns).

$$\left(\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 2 \\ 1 & -2 & -1 \end{bmatrix} * \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & -2 \\ 1 & 2 & -1 \end{bmatrix}\right)^{-1} * \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 2 \\ 1 & -2 & -1 \end{bmatrix} \begin{bmatrix} -0.8 \\ 0.1 \\ -5.0 \end{bmatrix} = \begin{bmatrix} 0.186 \\ -2.057 \\ 1.071 \end{bmatrix}$$

```
[40]: X = np.array([
          [1, 1, 1],
          [1, -1, -2],
          [1, 2, -1]
      ])

      xtx = X.T @ X
      xtx_inverse = np.linalg.inv(xtx)

      xty = X.T @ y

      w = xtx_inverse @ xty

      y_pred = X @ w

      errors = y_pred - y

      total_error = np.sum(errors ** 2)

      w, total_error

[40]: (array([ 0.18571429, -2.05714286,  1.07142857]), 1.4860475450891912e-30)
```

Our corresponding total error is: 1.4860475450891912e-30 (essentially zero).

Question 2

$y^{(i)} = x^{(i)} + \epsilon^{(i)}, \epsilon - N(0,1), x \in R$

-5	-6.2	-5.6
0	0.7	0.6
1	2.5	1.2
6	6.4	7.2

Compare a linear model: $\hat{y} = w_1 x$ to a polynomial model: $\hat{y} = w_1 x + w_2 x^2$

Part (a):

Linear model: Since the data is being generated by $y^{(i)} = x^{(i)} + \epsilon^{(i)}$, where ϵ is our noise with mean zero ($N(mean = 0, SD = 1)$), that means that y would on average = x. This is because when we have infinite data, the noise would average out since we would have infinity + and - noise, and our model would find the true fit between x and y. Therefore, our parameter would converge to 1. - The true function would be y = x (ignore noise)

Polynomial model: With this model, we can take the logic from the linear model and apply w_1 converging to 1. For our w_2x^2, our actual generating function does not have a quadratic equation or number. It is strictly liner, which would converge our second parameter to zero and still apply the same rules for our first parameter w_1 which converges to 1.

Part (b):

$$\text{X (linear)} = \begin{bmatrix} -5 \\ 0 \\ 1 \\ 6 \end{bmatrix}, \text{X (poly)} = \begin{bmatrix} -5 & 25 \\ 0 & 0 \\ 1 & 1 \\ 6 & 36 \end{bmatrix}, y_1 = \begin{bmatrix} -6.2 \\ 0.7 \\ 2.5 \\ 6.4 \end{bmatrix}, y_2 = \begin{bmatrix} -5.6 \\ 0.6 \\ 1.2 \\ 7.2 \end{bmatrix}$$

```
[59]: x1 = np.array([ [-5], [0], [1], [6] ])

      x2 = np.array([
          [-5, 25],
          [0, 0],
          [1, 1],
          [6, 36]
      ])
      #                                                    Equation: (xTx)^-1 xty
      y1 = np.array([ [-6.2], [0.7], [2.5], [6.4] ])
      y2 = np.array([ [-5.6], [0.6], [1.2], [7.2] ])

      # x1 | y1, y2
      x1tx1 = x1.T @ x1
      x1tx1_inverse = np.linalg.inv(x1tx1)

      x1ty1 = x1.T @ y1
      x1ty2 = x1.T @ y2
      w1 = x1tx1_inverse @ x1ty1
      w2 = x1tx1_inverse @ x1ty2

      # ------------------------------------ #

      # x2 | y1, y2
      x2tx2 = x2.T @ x2
      x2tx2_inverse = np.linalg.inv(x2tx2)

      x2ty1 = x2.T @ y1
      x2ty2 = x2.T @ y2
      w3 = x2tx2_inverse @ x2ty1
      w4 = x2tx2_inverse @ x2ty2

      w1, w2, w3, w4

[59]: (array([[1.15967742]]),
       array([[1.16774194]]),
       array([[ 1.18360434],
              [-0.01612466]]),
       array([[1.15696477],
              [0.00726287]]))
```

With our code above, we get our solutions after using the closed form solution. Our answers become:

- Linear fit for (x_1, y_1): 1.159
- Linear fit for (x_1, y_2): 1.167
- Polynomial fit for (x_1, y_1): [1.183, -0.016]
- Polynomial fit for (x_1, y_2): [1.156, 0.007]

Part (c):

change in w_1 for our linear model:

$(x_1, y_2) - (x_1, y_1) = 1.167 - 1.159 = 0.008$

Change in w_1 for our polynomial model:

$(x_1, y_2) - (x_1, y_1) = 1.183 - 1.156 = 0.027$

From this, we can see that our polynomial model had a larger change in w_1. This is because we have an extra parameter on our polynomial model, which means that small changes in our data lead to larger shifts in our parameters. Both parameters adjust to one another with shifting and fitting the data, which

can lead to larger changes. The linear model on the other hand, is what it says; linear, which makes it less flexible and more stable.

Question 3

$x(i) \in R$,

1	1	1
2	-1	0
-3	-1	1
-3	1	1

Part (a):

We can use the sigmoid function to simplify our Probability equation (from lecture slides).

$o(z) = \frac{1}{1+e^{-z}}$ where $z = w * x$

- log and e cancel each other out, $\log(e^x) = x_{(i)}$

$$P(y^{(i)} = +1|x^{(i)}; w) = \frac{1}{1+e^{-z}}$$

$$P(y^{(i)} = 0|x^{(i)}; w) = 1 - \frac{1}{1+e^{-z}}$$

for $z = w_{(i)} * x_{(i)}$:

$$\frac{\partial ll(w)}{\partial w_1} = \frac{1}{n}\sum_{i=1}^{n}[y^{(i)} - o(z) * x_1]$$

$$\frac{\partial ll(w)}{\partial w_2} = \frac{1}{n}\sum_{i=1}^{n}[y^{(i)} - o(z) * x_2]$$

Part (b):

```python
xx = np.array([
    [1, 1],
    [2, -1],
    [-3, -1],
    [-3, 1]
])

yy = np.array([1, 0, 1, 1])

w = np.array([0, 0])

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def average_log_likelihood(w):
    log_likelihood = 0
    for a, b in zip(xx, yy):
        z = w @ a
        p = sigmoid(z)
        log_likelihood += b * np.log(p) + (1 - b) * np.log(1 - p)
    return log_likelihood / len(yy)


def gradient_ascent(alpha, n = 4):
    guesses = [np.array([0, 0])]
    current_guess = guesses[0]
    while len(guesses) < n + 1:
        gradient = np.array([0.0, 0.0])
        for a, b in zip(xx, yy):
            z = current_guess @ a
            p = sigmoid(z)
            gradient += (b - p) * a
        gradient = gradient / len(yy)
        current_guess = current_guess + alpha * gradient
        guesses.append(current_guess)
    return np.array(guesses)
```

```python
iteration1 = gradient_ascent(0.01, n = 4)
iteration2 = gradient_ascent(0.2, n = 4)
w1 = iteration1[-1] #last element
w2 = iteration2[-1]
ll1 = average_log_likelihood(w1)
ll2 = average_log_likelihood(w2)
```

```python
print(f"iteration 1: {iteration1}, w1: {w1}, average log likelihood: {ll1}")
```

```
iteration 1: [[ 0.          0.        ]
 [-0.00875     0.0025    ]
 [-0.01737266  0.00498828]
 [-0.02586987  0.00746496]
 [-0.03424349  0.00993015]], w1: [-0.03424349  0.00993015], average log likelihood: -0.6615776273762636
```

```python
print(f"iteration 2: {iteration2}, w2: {w2}, average log likelihood: {ll2}")
```

```
iteration 2: [[ 0.          0.        ]
 [-0.175       0.05      ]
 [-0.30009838  0.09545861]
 [-0.39264224  0.13768578]
 [-0.46380564  0.17759553]], w2: [-0.46380564  0.17759553], average log likelihood: -0.3954420670938938
```

Part (c):

After running 4 iterations with the code given above, the learning rate of alpha = 0.2 is larger than the learning rate of alpha = 0.01. To be more specific, our 0.01 alpha has a higher average log likelihood of -0.66 compared to our 0.2 alpha which has a average log likelihood of -0.39. This means